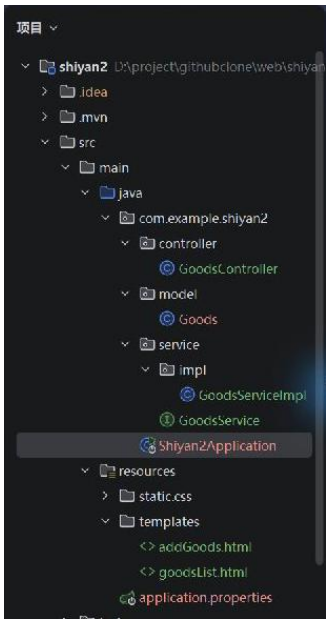
浙江树人学院 **信息科技** 学院

实 验 报 告

课　程	**Web 开发技术**	实验名称	Thymeleaf 模板引擎 商品信息管理
班　级		姓　　名	
日　期	2026 年 10 月 9 日	学　　号	

实验目的	1．掌握 Spring Boot 集成 Thymeleaf 的方法，了解模板位置、视图名称与访问路径的关系。 2．掌握 th:object 与 th:field 的表单绑定，以及 th:each 与 th:text 的列表渲染。 3．理解 Controller、Goods 模型和 Service 的协作方式，使用静态集合模拟商品存储，并如实记录实现与运行结果。
实验内容	**一 实验过程** **1 创建工程与组织目录** 本实验在 shiyan2 工程中实现商品添加与列表显示。工程按 controller、model、service 分层；模板放在 src/main/resources/templates，样式文件为 static/css/goods.css。控制器处理请求，Service 保存商品，Thymeleaf 将模型数据渲染成 HTML。  图 1 IDEA 中的工程目录结构 图 1 中可见 GoodsController、Goods、GoodsService、GoodsServiceImpl 及两个模板文件。启动类 Shiyan2Application 位于 com.example.shiyan2 包，能够扫描其下的组件。

2 配置 Maven 依赖与运行环境

pom.xml 使用 Spring Boot 4.1.1，java.version 为 17。spring-boot-starter-webmvc 提供 Web MVC 能力，spring-boot-starter-thymeleaf 用于服务端模板渲染。工程使用 Maven Wrapper，因此构建时执行项目内的 mvnw.cmd。

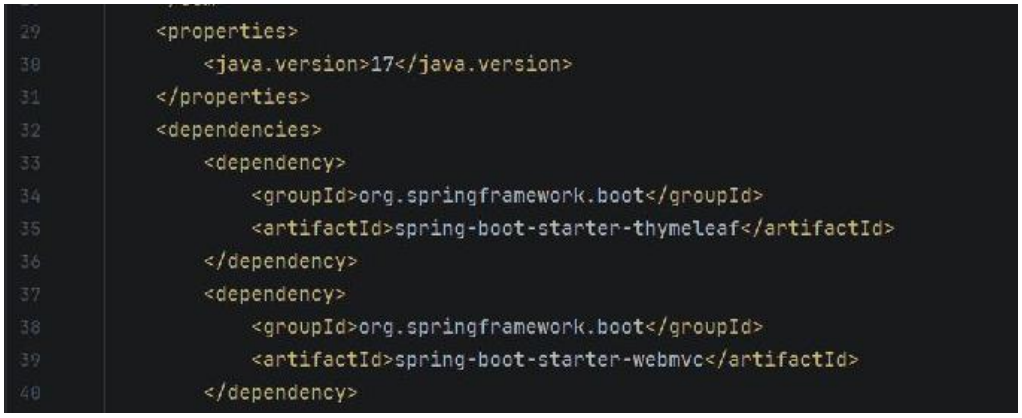


图 2 IDEA 中的 Java 版本和 Web 及 Thymeleaf 依赖

3 使用 Goods 模型封装参数

Goods 包含 id、goodsName、originalPrice、discountPrice 四个属性。编号使用 Long，名称使用 String，两个价格使用 BigDecimal，并初始化为 0.0。无参构造方法和属性的 getter、setter 支持 Spring MVC 参数绑定与模板读取。



图 3 IDEA 中的 Goods 核心字段与构造方法

表单提交的字段名与 Goods 的属性名一致，Spring MVC 将名称和价格封装为一个 Goods 对象。编号不由页面输入，而由 Service 在保存时分配。这样控制器不必逐项读取请求参数。

4 在 Service 中模拟商品存储

GoodsService 定义 addGoods(Goods goods) 和 findAll() 两个方法。GoodsServiceImpl 通过 @Service 注册为 Spring 组件，使用静态 List<Goods> 集合 GOODS_LIST 保存商品，并使用静态 nextId 从 1 开始生成编号。

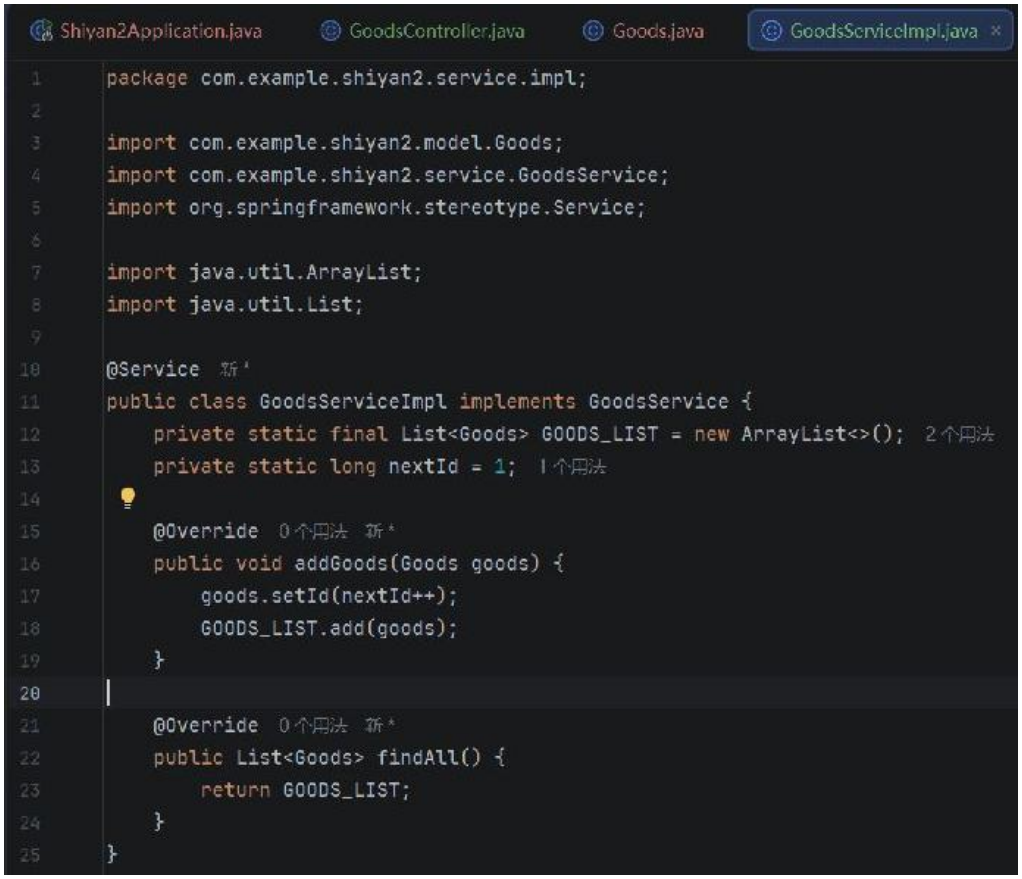


图 4 IDEA 中的 Service 实现与静态集合

添加时先执行 goods.setId(nextId++)，再将对象加入集合；findAll() 返回当前集合。静态集合在同一应用进程内保留先后提交的数据，使后续列表能够显示已经添加的商品。

该方式没有连接数据库，也没有写入文件。应用进程结束或重启后，原集合中的数据会丢失，编号重新从 1 开始，适合本实验的模拟存储要求。

5 编写控制器并注入 Service

GoodsController 使用 @Controller 声明控制器，在类上用 @RequestMapping("/shiyan1") 指定路径前缀，通过 @Autowired 注入 GoodsService。控制器只有 toAdd 和 addGoods 两个方法，符合实验要求。

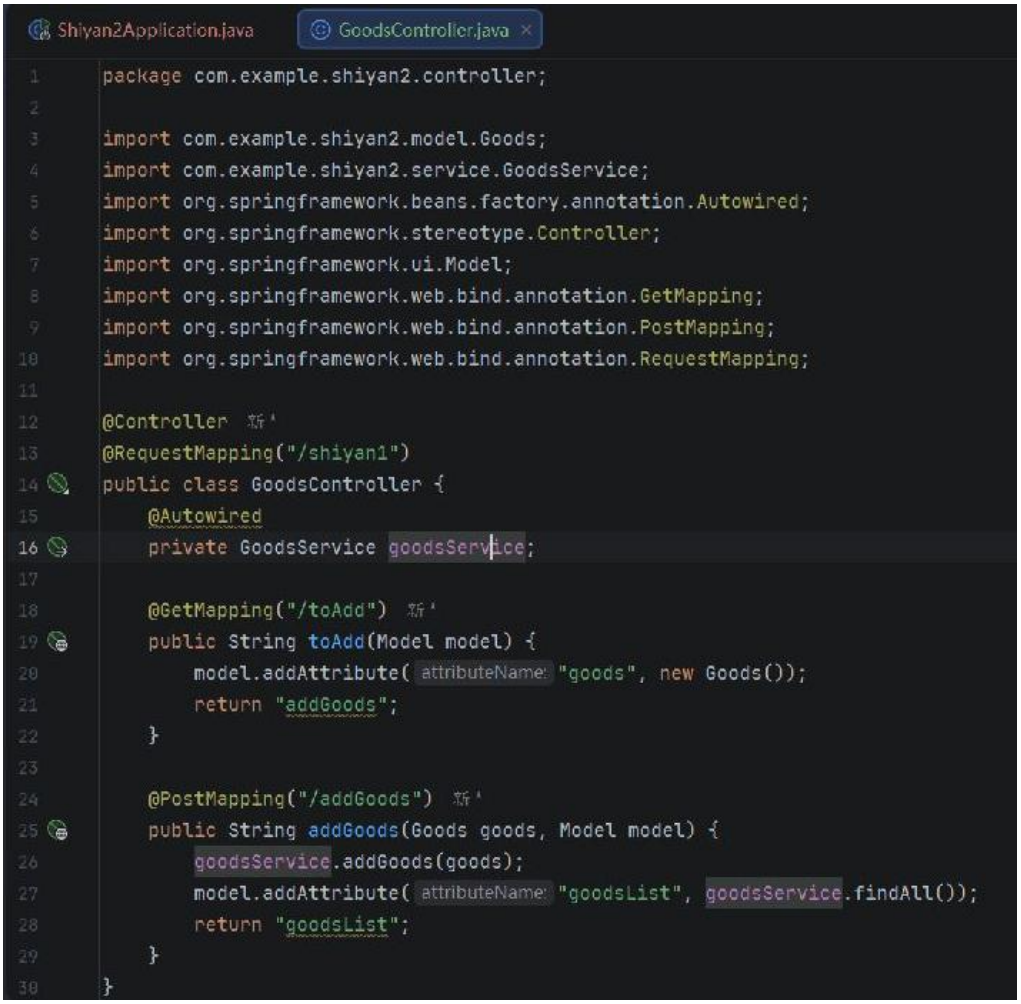


图 5 IDEA 中的完整 GoodsController

toAdd 处理 GET /shiyan1/toAdd，在 Model 中添加名为 goods 的新对象，返回视图名 addGoods。模板引擎据此查找 templates/addGoods.html，并用对象的初始值渲染表单。

addGoods 处理 POST /shiyan1/addGoods，使用 Goods 参数接收表单数据，调用 Service 保存，然后将 findAll() 的结果以 goodsList 为名加入 Model，返回视图名 goodsList。请求路径、模型属性名与模板表达式需要对应。

6 实现添加页的表单绑定

addGoods.html 使用 th:action="@{/shiyan1/addGoods}" 设置提交地址，method="post" 指定提交方式，th:object="${goods}" 指定绑定对象。三个输入框分别使用 *{goodsName}、*{originalPrice} 和 *{discountPrice}。

```
<form class="goods-form" th:action="@{/shiyan1/addGoods}" th:object="${goods}" method=
    <div class="goods-form__row">
        <label for="goodsName">商品名称</label>
        <input id="goodsName" type="text" th:field="*{goodsName}" placeholder="请输入商
    </div>

    <div class="goods-form__row">
        <label for="originalPrice">商品原价</label>
        <input id="originalPrice" type="number" th:field="*{originalPrice}" step="0.01
    </div>

    <div class="goods-form__row">
        <label for="discountPrice">商品折扣价</label>
        <input id="discountPrice" type="number" th:field="*{discountPrice}" step="0.01
    </div>

    <div class="goods-form__actions">
        <span aria-hidden="true"></span>
        <div>
            <button class="button button--add" type="submit">添加</button>
            <button class="button button--reset" type="reset">重置</button>
        </div>
```

图 6 IDEA 中的添加页核心绑定与按钮片段

商品名称设置 required；价格输入框采用 number 类型、step="0.01"、min="0"。添加按钮为 submit，重置按钮为 reset。required 和 min 属于浏览器端约束，当前实现尚未增加独立的服务端业务校验。

7 实现商品列表的遍历与输出

goodsList.html 在表格行上使用 th:each="item : ${goodsList}" 遍历集合，再以 th:text 输出 item.id、item.goodsName、item.originalPrice 和 item.discountPrice，每个商品对应一行。

```
    <table class="goods-table">
        <tbody>
        <tr th:each="item : ${goodsList}">
            <td th:text="${item.id}"></td>
            <td th:text="${item.goodsName}"></td>
            <td th:text="${item.originalPrice}"></td>
            <td th:text="${item.discountPrice}"></td>
        </tr>
        </tbody>
    </table>
</main>
</body>
</html>
```

图 7 IDEA 中的列表遍历与字段输出片段

8 启动应用并执行上下文测试

在 IDEA 中运行 Shiyan2Application。实际启动日志显示 Java 17.0.12、Spring Boot 4.1.1，Tomcat 启动于 8080 端口，并出现 Started Shiyan2Application，说明本次实例完成启动。



图 8a IDEA 运行窗口与 Spring Boot 版本

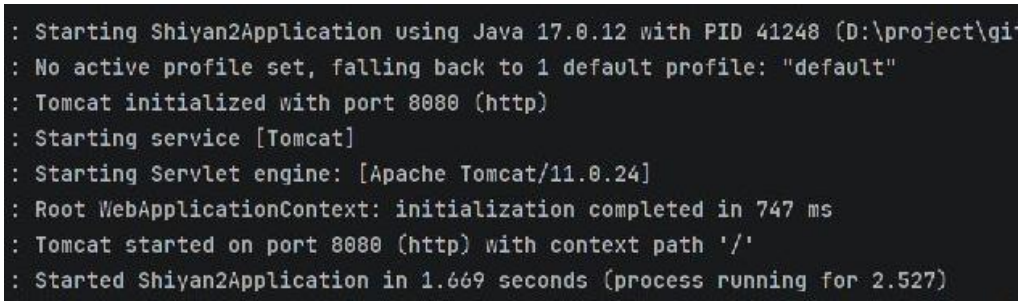


图 8b IDEA 启动日志中的 Java 版本与 8080 端口

2026 年 10 月 9 日 11:50，在 PowerShell 中将 JAVA_HOME 设为 D:\pc\.jdks\graalvm-jdk-17.0.12，执行 .\mvnw.cmd -B test。结果为 BUILD SUCCESS；Tests run: 1，Failures: 0，Errors: 0，Skipped: 0。

现有 Shiyan2ApplicationTests 只有 contextLoads()，验证 Spring 上下文加载。浏览器功能以实际操作及截图核对，不能仅凭这一个测试认定完整业务流程已经验证。

二 实验结果及分析

添加页入口为 http://localhost:8080/shiyan1/toAdd。虽然工程名称是 shiyan2，但控制器路径前缀按作业要求保留 /shiyan1，两者的用途不同。



图 9 浏览器中的添加商品页面

1 核对提交后的商品列表

本次添加两条商品：名称“1”，原价 12、折扣价 12；名称“2”，原价 13、折扣价 14。提交后的地址为 /shiyan1/addGoods，列表显示编号 1、2 及对应名称、价格（图 10），说明参数已传入列表并完成渲染。

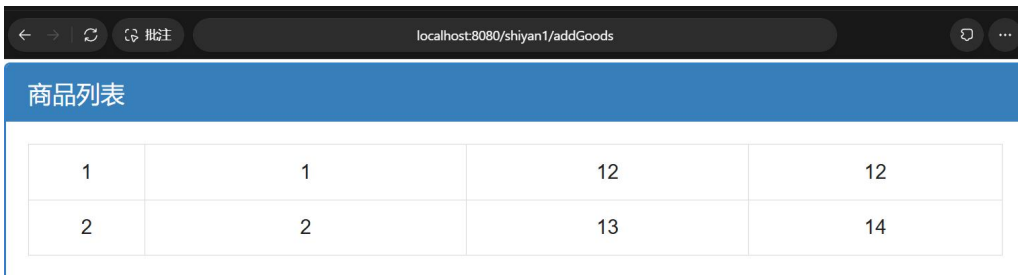


图 10 浏览器中两次提交后的商品列表

2 分析数据绑定与视图渲染

商品信息经过“表单输入→Goods 参数绑定→Service 集合保存→Model 属性→Thymeleaf 列表渲染”的流程。goods 是添加页绑定对象的名称，goodsList 是列表页集合属性的名称；如果这些名称不一致，模板就无法正确取得数据。

重置验证：修改三个输入框后单击“重置”，名称恢复为空、两个价格恢复为 0.0（图 9），结果正常。重置恢复当前表单的初始值，不会删除 Service 集合中的商品。

当前功能实现了添加与展示，静态集合保留的是进程内状态。价格由 BigDecimal 接收，当前 th:text 按对象内容直接显示，并未增加统一保留两位小数的格式化规则。

第二条商品的折扣价 14 高于原价 13 仍可显示，与当前代码未检查两种价格之间的大小关系一致。若以后增加实际业务要求，应在 Service 层补充价格校验。

心得体会	**一 心得体会** 本实验让我理解了 MVC 的职责划分：控制器接收请求并选择视图，Goods 表示商品数据，Service 保存和读取商品，模板负责展示。保存逻辑放在 Service 中，使控制器更清楚，也便于以后改用数据库。 表单的 th:field、Goods 属性和请求参数名称需要对应，列表的 goodsList 也要与 Model 属性一致。排查页面问题时，应依次核对 URL 映射、视图名称、模板位置和绑定表达式。 **二 问题分析** **问题一 访问根路径出现 404** 问题描述：访问 http://localhost:8080 出现 Whitelabel Error Page，提示 404。 问题分析：控制器只映射 /shiyan1/toAdd 和 /shiyan1/addGoods，未定义根路径 /。 解决方法：访问 http://localhost:8080/shiyan1/toAdd，使用类级与方法级映射组成的完整入口。 **问题二 视图名称与模板文件未对应** 问题描述与分析：先前缺少所需模板。控制器返回视图名称后，Thymeleaf 需要找到相应 HTML 文件，才能形成完整页面。 解决方法：在 src/main/resources/templates 补齐 addGoods.html 和 goodsList.html，核对视图名、模型属性和样式路径。当前目录与模板代码见图 1、图 6、图 7。
成绩评定	指导老师：